# FluxDone

Technical Requirements Document (TRD)

*Version 1.0 | March 2026 | Flux App Family*

# 1. Document Overview

| | |
|---|---|
| **App Name** | FluxDone (FD) |
| **Document Type** | Technical Requirements Document (TRD) |
| **Version** | 1.0 |
| **Companion Document** | FluxDone Product Requirements Document (PRD) v1.0 |
| **App Family** | Flux — FluxDone (task management) + FluxFocus (screen discipline) |
| **Primary Platform** | Android (API 26 / Android 8.0 minimum) |
| **Future Platforms** | iOS, Web (Phase 2) |
| **Date** | March 2026 |

This document defines the full technical architecture, technology stack, database schema, module boundaries, API design, performance requirements, deployment strategy, and development checklist for FluxDone. It is intended to be the primary reference for AI-assisted code generation. Every section of this document must be read and understood before any code is written. Deviations from this document require explicit approval and must be documented.

# 2. System Architecture

## 2.1 Architecture Pattern

FluxDone uses a layered, feature-first modular architecture. The application is divided into independent feature modules. Each feature module owns its own UI layer, business logic layer, and data layer. No feature module may directly import from another feature module's internal files. Cross-module communication happens exclusively through a shared service layer or a pub/sub event bus.

## 2.2 Layer Definitions

| Layer | Responsibility |
|---|---|
| **Presentation Layer** | Flutter Widgets, Screens, and UI state. Owns all visual rendering. Communicates exclusively with the Domain Layer via ViewModels or BLoC/Cubit state classes. |
| **Domain Layer** | Business logic, use cases, and entity definitions. Framework-independent. Contains the rules for what the app does (e.g., “creating a task requires a non-empty title and a valid date”). Does not import Flutter or any database package. |
| **Data Layer** | Repository implementations, local database access (SQLite/Hive), and external API clients (Google Calendar, Google Drive). Implements interfaces defined in the Domain Layer. |
| **Shared / Core** | App-wide utilities: theme definitions, color constants, date formatters, navigation router, event bus, dependency injection container, and permission handlers. |

## 2.3 Module Boundaries

The following feature modules are defined. Each is a self-contained folder inside lib/features/. No module may access another module’s internal files directly.

| Module | Owns | Exposes to Other Modules |
|---|---|---|
| `tasks` | Task CRUD, subtasks, priority, recurrence, reminders | *TaskCreatedEvent, TaskUpdatedEvent, TaskDeletedEvent (via event bus)* |
| `calendar` | Timeline rendering, calendar view modes, Google Calendar overlay | *None — reads from tasks module via shared repository interface* |
| `habits` | Habit CRUD, streak logic, completion history | *None* |
| `lists` | Folder/List/Section management, custom colors | *ListColorProvider (used by tasks and calendar for color lookup)* |
| `smart_lists` | Today, Tomorrow, Upcoming, All, Completed, Trash query logic | *None — reads from tasks repository* |
| `navigation` | Route definitions, hamburger drawer, bottom/left nav bar | *AppRouter, NavigationService* |
| `settings` | Theme toggle, Google account link, notification permissions | *ThemeNotifier, GoogleAuthService* |
| `backup` | Google Drive DB upload/download, backup scheduling | *BackupService* |
| `fluxfocus_bridge` | Inter-app Intent / IPC communication with FluxFocus | *FocusBlockRequest* |

## 2.4 Folder Structure

```
lib/
  main.dart                     # App entry point, DI initialization
  app.dart                      # MaterialApp, theme, router setup
  core/
    theme/                      # Light and dark ThemeData definitions
    colors/                     # App-wide color constants + domain color map
    router/                     # GoRouter route definitions
    events/                     # Event bus definitions (TaskCreatedEvent,
etc.)
    utils/                      # Date formatters, string helpers
    permissions/                # Android permission request wrappers
  features/
    tasks/
      data/                     # TaskDao, TaskRepository implementation
      domain/                   # Task entity, ITaskRepository interface, use
cases
      presentation/             # TaskListScreen, TaskDetailSheet, TaskCard
widget
    calendar/
      data/                     # GoogleCalendarApiClient
      domain/                   # CalendarEvent entity, ICalendarRepository
      presentation/             # CalendarScreen, TimelineWidget, TaskBlock
widget
    habits/
      data/                     # HabitDao, HabitRepository implementation
      domain/                   # Habit entity, StreakCalculator, use cases
      presentation/             # HabitTrackerScreen, HabitCard,
HabitDetailScreen
    lists/
      data/                     # FolderDao, ListDao, SectionDao
      domain/                   # Folder, TaskList, Section entities
      presentation/             # ListManagementScreen, ColorPickerWidget
    smart_lists/
      domain/                   # SmartListQueryService
      presentation/             # SmartListScreen (reuses TaskCard from tasks
module)
    navigation/
      presentation/             # AppDrawer, BottomNavBar, LeftNavRail
    settings/
      presentation/             # SettingsScreen
      data/                     # LocalPreferencesRepository
(shared_preferences)
    backup/
      data/                     # GoogleDriveApiClient, BackupRepository
      presentation/             # BackupScreen
```

```
  fluxfocus_bridge/
    data/                    # AndroidIntentService, IPCService
```

# 3. Technology Stack

| Category | Technology | Rationale |
|---|---|---|
| **UI Framework** | `Flutter (Dart)` | Single codebase for Android → iOS → Web. Dart is readable for non-coders directing AI. Strong widget ecosystem. |
| **State Management** | `flutter_bloc (BLoC / Cubit)` | Clear separation of UI and logic. Predictable state. Well-documented for AI code generation. |
| **Local Database** | `sqflite (SQLite)` | Relational structure suits tasks, lists, folders, and habits. ACID-compliant. Offline-first by nature. |
| **Dependency Injection** | `get_it + injectable` | Service locator pattern. Keeps module dependencies explicit and testable. |
| **Navigation** | `go_router` | Declarative routing. Deep-link ready for Phase 2. Supports nested navigation for drawer + bottom nav. |
| **Local Notifications** | `flutter_local_notifications` | On-device reminder delivery. No network required. Full Android notification channel support. |
| **Google Sign-In** | `google_sign_in` | OAuth 2.0 authentication. Required for Google Calendar and Google Drive features. |
| **Google Calendar API** | `googleapis (calendar_v3)` | Read-only event fetch. Used for overlay in Calendar View. |
| **Google Drive API** | `googleapis (drive_v3)` | DB file upload/download for cross-device backup. |
| **Key-Value Storage** | `shared_preferences` | Lightweight storage for app settings: theme, linked Google account, last backup timestamp. |
| **Inter-App IPC** | `Android Intents (via platform channel)` | FluxDone → FluxFocus communication. Uses Flutter's MethodChannel to invoke native Android Intent. |
| **Date/Time** | `intl package` | Locale-aware date and time formatting across all screens. |
| **Testing** | `flutter_test + mocktail` | Unit tests for domain logic. Widget tests for critical UI flows. |

# 4. Database Schema

## 4.1 Schema Design Principles

- All primary keys are auto-incrementing integers (INTEGER PRIMARY KEY AUTOINCREMENT).
- All timestamps are stored as INTEGER (Unix epoch milliseconds, UTC). The application layer converts to local time for display.
- Boolean fields are stored as INTEGER (0 = false, 1 = true).
- Nullable fields are explicitly marked. Non-nullable fields have NOT NULL constraints.
- Soft deletes are used for tasks (is_trashed flag) and completed tasks (is_completed flag). Hard deletes are used only for permanent trash emptying.
- Recurrence rules are stored as a JSON string in a single column to avoid a separate recurrence table for Phase 1.
- All foreign keys use ON DELETE CASCADE to maintain referential integrity.

## 4.2 Table: folders

| Column | Type | Constraints | Description |
|---|---|---|---|
| `id` | `INTEGER` | `PRIMARY KEY AUTOINCREMENT` | Unique folder identifier |
| `name` | `TEXT` | `NOT NULL` | Display name of the folder |
| `sort_order` | `INTEGER` | `NOT NULL DEFAULT 0` | User-defined sort position in the sidebar |
| `created_at` | `INTEGER` | `NOT NULL` | Creation timestamp (Unix ms, UTC) |
| `updated_at` | `INTEGER` | `NOT NULL` | Last modification timestamp (Unix ms, UTC) |

## 4.3 Table: task_lists

| Column | Type | Constraints | Description |
|---|---|---|---|
| `id` | `INTEGER` | `PRIMARY KEY AUTOINCREMENT` | Unique list identifier |
| `folder_id` | `INTEGER` | `NOT NULL, FK → folders(id) ON DELETE CASCADE` | Parent folder |
| `name` | `TEXT` | `NOT NULL` | Display name of the list |
| `color_hex` | `TEXT` | `NOT NULL` | 6-character hex color string without # prefix (e.g., 2E7D32) |
| `sort_order` | `INTEGER` | `NOT NULL DEFAULT 0` | User-defined sort position within its folder |

| Column | Type | Constraints | Description |
|---|---|---|---|
| created_at | INTEGER | NOT NULL | Creation timestamp (Unix ms, UTC) |
| updated_at | INTEGER | NOT NULL | Last modification timestamp (Unix ms, UTC) |

## 4.4 Table: sections

| Column | Type | Constraints | Description |
|---|---|---|---|
| id | INTEGER | PRIMARY KEY AUTOINCREMENT | Unique section identifier |
| list_id | INTEGER | NOT NULL, FK → task_lists(id) ON DELETE CASCADE | Parent list |
| name | TEXT | NOT NULL | Display name of the section (e.g., Physics, Chemistry) |
| sort_order | INTEGER | NOT NULL DEFAULT 0 | Position within the list |
| created_at | INTEGER | NOT NULL | Creation timestamp (Unix ms, UTC) |

## 4.5 Table: tasks

| Column | Type | Constraints | Description |
|---|---|---|---|
| id | INTEGER | PRIMARY KEY AUTOINCREMENT | Unique task identifier |
| list_id | INTEGER | NOT NULL, FK → task_lists(id) ON DELETE CASCADE | Parent list |
| section_id | INTEGER | NULLABLE, FK → sections(id) ON DELETE SET NULL | Parent section (optional) |
| title | TEXT | NOT NULL | Task title. Cannot be empty. |
| description_md | TEXT | NULLABLE | Rich text description stored as markdown string |
| task_date | INTEGER | NOT NULL | Date the task belongs to (Unix ms, UTC, time component zeroed) |
| start_time | INTEGER | NULLABLE | Optional start time as Unix ms (UTC). Enables timeline block rendering. |

| Column | Type | Constraints | Description |
|---|---|---|---|
| end_time | INTEGER | NULLABLE | Optional end time as Unix ms (UTC). Required if start_time is set. |
| is_completed | INTEGER | NOT NULL DEFAULT 0 | 1 if the task is marked complete, 0 otherwise |
| completed_at | INTEGER | NULLABLE | Timestamp when the task was completed (Unix ms, UTC) |
| is_trashed | INTEGER | NOT NULL DEFAULT 0 | 1 if the task is in the Trash, 0 otherwise |
| trashed_at | INTEGER | NULLABLE | Timestamp when the task was trashed (Unix ms, UTC). Used for 30-day auto-purge. |
| priority | INTEGER | NOT NULL DEFAULT 0 | 0 = None, 1 = Low, 2 = Medium, 3 = High |
| recurrence_rule | TEXT | NULLABLE | JSON string encoding recurrence settings. Schema defined in section 4.8. |
| recurrence_parent_id | INTEGER | NULLABLE, FK → tasks(id) ON DELETE CASCADE | For recurring instances: references the original parent task. |
| sort_order | INTEGER | NOT NULL DEFAULT 0 | User-defined position within the list/section |
| created_at | INTEGER | NOT NULL | Creation timestamp (Unix ms, UTC) |
| updated_at | INTEGER | NOT NULL | Last modification timestamp (Unix ms, UTC) |

## 4.6 Table: subtasks

| Column | Type | Constraints | Description |
|---|---|---|---|
| id | INTEGER | PRIMARY KEY AUTOINCREMENT | Unique subtask identifier |
| task_id | INTEGER | NOT NULL, FK → tasks(id) ON DELETE CASCADE | Parent task |
| title | TEXT | NOT NULL | Subtask display text |
| is_completed | INTEGER | NOT NULL DEFAULT 0 | 1 if completed, 0 otherwise |
| sort_order | INTEGER | NOT NULL DEFAULT 0 | Position within the parent task's subtask list |
| created_at | INTEGER | NOT NULL | Creation timestamp (Unix ms, UTC) |

## 4.7 Table: reminders

| Column | Type | Constraints | Description |
|---|---|---|---|
| `id` | INTEGER | PRIMARY KEY AUTOINCREMENT | Unique reminder identifier |
| `task_id` | INTEGER | NOT NULL, FK → tasks(id) ON DELETE CASCADE | Parent task |
| `remind_at` | INTEGER | NOT NULL | Absolute timestamp for notification fire (Unix ms, UTC) |
| `offset_minutes` | INTEGER | NULLABLE | If set, reminder was defined as N minutes before task start. Stored for recurrence regeneration. |
| `notification_id` | INTEGER | NOT NULL | Unique ID passed to flutter_local_notifications for cancellation |

## 4.8 Table: habits

| Column | Type | Constraints | Description |
|---|---|---|---|
| `id` | INTEGER | PRIMARY KEY AUTOINCREMENT | Unique habit identifier |
| `name` | TEXT | NOT NULL | Display name of the habit |
| `color_hex` | TEXT | NOT NULL | 6-character hex color string without # prefix |
| `frequency_rule` | TEXT | NOT NULL | JSON string. Schema: {type: 'daily'\|'specific_days'\|'x_per_week', days: [0..6], times_per_week: N} |
| `current_streak` | INTEGER | NOT NULL DEFAULT 0 | Current consecutive completion streak count |
| `longest_streak` | INTEGER | NOT NULL DEFAULT 0 | All-time longest streak achieved |
| `is_active` | INTEGER | NOT NULL DEFAULT 1 | 1 if habit is active, 0 if archived |
| `created_at` | INTEGER | NOT NULL | Creation timestamp (Unix ms, UTC) |

## 4.9 Table: habit_completions

| Column | Type | Constraints | Description |
|---|---|---|---|
| `id` | INTEGER | PRIMARY KEY AUTOINCREMENT | Unique completion record identifier |
| `habit_id` | INTEGER | NOT NULL, FK → habits(id) ON DELETE CASCADE | Parent habit |
| `completed_date` | INTEGER | NOT NULL | Date of completion (Unix ms, UTC, time zeroed). UNIQUE per habit_id. |

### 4.10 Recurrence Rule JSON Schema

The recurrence_rule column in the tasks table stores a JSON string. The following schema defines all valid structures:

```
{ "type": "daily" }
{ "type": "weekly", "days": [1, 3, 5] }          // 0=Sun, 1=Mon ... 6=Sat
{ "type": "monthly_date", "day": 15 }            // Every 15th of the month
{ "type": "monthly_weekday", "week": 2, "day": 1 } // Every 2nd Monday
{ "type": "interval", "every_n_days": 3 }        // Every 3 days
```

When a recurring task instance is modified, the edit_scope field (passed at the application layer, not stored) determines the mutation behavior:

- this_only — Detach this instance from the recurrence chain (set recurrence_parent_id to null, clear recurrence_rule) and apply changes.
- this_and_future — Update this instance and regenerate all future instances from this date forward.
- all — Update the parent task and regenerate all past and future instances.

## 5. API Design

### 5.1 Internal Repository Interface Contracts

All feature modules access data exclusively through repository interfaces defined in the domain layer. The data layer provides concrete implementations. This enables future swapping of the storage backend without touching business logic or UI.

#### ITaskRepository

```
Future<List<Task>> getTasksByListId(int listId)
Future<List<Task>> getTasksByDate(DateTime date)
Future<List<Task>> getTasksByDateRange(DateTime start, DateTime end)
Future<List<Task>> getCompletedTasks()
```

```
Future<List<Task>> getTrashedTasks()
Future<Task> getTaskById(int id)
Future<int> createTask(Task task)
Future<void> updateTask(Task task)
Future<void> trashTask(int id)
Future<void> restoreTask(int id)
Future<void> permanentlyDeleteTask(int id)
Future<void> emptyTrash()
Future<List<Task>> getRecurringInstances(int parentId)
```

### IListRepository

```
Future<List<Folder>> getAllFolders()
Future<List<TaskList>> getListsByFolderId(int folderId)
Future<List<Section>> getSectionsByListId(int listId)
Future<int> createFolder(Folder folder)
Future<int> createList(TaskList list)
Future<int> createSection(Section section)
Future<void> updateFolder(Folder folder)
Future<void> updateList(TaskList list)
Future<void> deleteFolder(int id)
Future<void> deleteList(int id)
Future<String> getColorHexByListId(int listId)
```

### IHabitRepository

```
Future<List<Habit>> getAllActiveHabits()
Future<Habit> getHabitById(int id)
Future<int> createHabit(Habit habit)
Future<void> updateHabit(Habit habit)
Future<void> archiveHabit(int id)
Future<void> recordCompletion(int habitId, DateTime date)
Future<void> removeCompletion(int habitId, DateTime date)
Future<List<DateTime>> getCompletionHistory(int habitId)
```

## 5.2 Google Calendar API Usage

- Endpoint: GET https://www.googleapis.com/calendar/v3/calendars/primary/events
- Authentication: OAuth 2.0 Bearer token obtained via google_sign_in package.
- Query parameters: timeMin (start of visible calendar range), timeMax (end of visible calendar range), singleEvents=true, orderBy=startTime.
- Response is mapped to a CalendarEvent entity containing: id, title, start datetime, end datetime, color (if set by user in Google Calendar).

- Fetched events are cached in memory for the current calendar view session. Cache is invalidated when the user changes the date range or manually refreshes.
- All Google Calendar API calls are wrapped in try/catch. Failures are silent — the calendar timeline renders without the overlay rather than blocking.

### 5.3 Google Drive API Usage (Phase 2)

- Authentication: Same OAuth 2.0 session as Google Calendar (shared google_sign_in instance, extended with drive.file scope).
- On first backup: Create a folder named FluxDone_Backup in the user's Google Drive root. Store the folder ID in shared_preferences.
- Backup operation: Upload the local SQLite database file (fluxdone.db) to the FluxDone_Backup folder as a binary file. If a file with the same name already exists, replace it (update, not duplicate).
- Restore operation: Download the latest fluxdone.db from the FluxDone_Backup folder and replace the local database file. Requires app restart after restore.
- Backup is triggered manually by the user from the Backup screen. Automatic scheduled backup is a Phase 2 enhancement, not Phase 1.

## 6. Security & Data Integrity

- No user data leaves the device in Phase 1. The app operates in a fully sandboxed local environment.
- In Phase 2, all network transmissions to Google APIs use HTTPS exclusively. Certificate pinning is not required for Phase 1 or 2.
- OAuth 2.0 tokens are managed by the google_sign_in package and stored in the platform's secure credential store (Android Keystore). The app never handles raw OAuth tokens directly.
- The SQLite database file is stored in the app's private internal storage directory (not accessible to other apps without root). No additional encryption is applied in Phase 1.
- The Google Drive scope requested is drive.file, which limits access to only files created by FluxDone. The app cannot read or write any other files in the user's Drive.
- All database write operations use parameterized queries via sqflite's built-in API. Raw SQL string concatenation is prohibited to prevent SQL injection.
- Reminder notification payloads contain only the task ID. Task content is retrieved from the local DB when the notification is tapped, never stored in the notification payload.

## 7. Deployment Strategy

### 7.1 Phase 1 — Local Development & Sideloading

- The app is built and deployed as a debug APK during development via flutter run or flutter build apk.

- Testing is performed on a physical Android device (developer's primary phone) and Android Emulator (API 26+ AVD).
- No Google Play Store submission in Phase 1. Distribution is via direct APK sideloading.
- Version naming: 0.x.y where x = phase (0 = pre-release), y = build increment.

### 7.2 Phase 2 — Multi-Device & Potential Publication

- Release APK built via flutter build apk --release with code obfuscation enabled.
- If publishing to Google Play Store: app signing via Play App Signing, privacy policy required (data handling disclosure for Google account access).
- iOS build requires Apple Developer account and Xcode. Flutter iOS target added in Phase 2.
- Web build via flutter build web. Hosted on Firebase Hosting or equivalent static host.

## 8. Performance Requirements

| Operation | Target | Measurement Method |
|---|---|---|
| App cold start to interactive | **< 2 seconds** | *Flutter DevTools timeline* |
| Task creation (tap FAB to task saved) | **< 200ms** | *Manual stopwatch on device* |
| List view render (up to 100 tasks) | **< 100ms** | *Flutter DevTools frame rate* |
| Calendar timeline initial render | **< 300ms** | *Flutter DevTools timeline* |
| Calendar timeline scroll (60fps) | **No frame drops below 50fps** | *Flutter DevTools frame rate* |
| Smart list query (All tasks) | **< 150ms for up to 500 tasks** | *sqflite query timing* |
| Google Calendar event fetch | **< 3 seconds on 4G** | *Network timing (non-blocking)* |
| Google Drive backup upload | **< 30 seconds for < 10MB DB** | *Background operation timer* |
| Local notification delivery | **Within 1 second of scheduled time** | *Manual device testing* |

## 9. Cost Estimate

FluxDone Phase 1 has zero monetary cost. All technologies used are free and open-source.

- Flutter SDK: Free, open-source (BSD license).
- All Dart packages (flutter_bloc, sqflite, go_router, etc.): Free, pub.dev open-source packages.
- Android development: Free. No Apple Developer account required until iOS phase.

- Google Calendar API: Free up to 1,000,000 requests per day. Personal use will never approach this limit.
- Google Drive API: Free up to 15GB storage per Google account (shared with existing Drive usage). DB backup file will be well under 10MB.
- Google Sign-In: Free for any use.
- Phase 2 potential costs: Apple Developer Program ($99/year USD) if publishing to App Store. Firebase Hosting free tier is sufficient for web build.

## 10. Development Checklist

### Phase 1 — P0 Features

*Complete these in order. Do not begin the next item until the current one passes user review on a physical device.*

- ☐ Flutter project initialized with correct folder structure (lib/features/, lib/core/).
- ☐ Dependency injection container (get_it) configured. All repositories registered.
- ☐ SQLite database initialized with all Phase 1 tables (folders, task_lists, sections, tasks, subtasks, reminders, habits, habit_completions).
- ☐ Database migration system in place (version-based schema upgrades).
- ☐ Folder CRUD operations implemented and tested (IListRepository → ListDao).
- ☐ List CRUD operations with custom color hex implemented and tested.
- ☐ Task CRUD operations implemented and tested (ITaskRepository → TaskDao).
- ☐ Task creation with mandatory title and date, optional start/end time, recurrence rule, and description_md.
- ☐ Task completion toggle (is_completed flag, completed_at timestamp).
- ☐ Task soft-delete to Trash (is_trashed flag, trashed_at timestamp).
- ☐ Trash auto-purge for tasks older than 30 days (run on app start).
- ☐ Smart list queries implemented: Today, Tomorrow, Upcoming (7 days), All, Completed, Trash.
- ☐ List View screen (SCR-01) renders tasks grouped by Section, with colored left-border accent.
- ☐ Task Detail / Edit Sheet (SCR-02) opens on task tap with all fields editable.
- ☐ Task Creation Sheet (SCR-03) opens from FAB with context pre-population.
- ☐ Rich text description editor with markdown rendering (H1-H3, bold, italic, lists).
- ☐ Recurring task instance generation logic implemented (daily, weekly, monthly_date, monthly_weekday, interval).
- ☐ Recurrence edit scope dialog (this only / this and future / all) implemented.
- ☐ Calendar View (SCR-04) renders vertical timeline for single-day, 3-day, and weekly modes.
- ☐ Calendar timeline: tapping empty slot opens Task Creation Sheet with pre-populated start time.
- ☐ Calendar timeline: overlapping task blocks rendered side-by-side (column layout).

- ☐ Calendar timeline: task blocks colored by their source list color.
- ☐ Hamburger drawer (SCR-08) opens on hamburger button tap, shows all Folders, Lists, and Smart Lists.
- ☐ Bottom navigation bar switches between List View, Calendar View, Habit Tracker, and Settings in portrait.
- ☐ Left navigation rail renders correctly in landscape orientation.
- ☐ App theme: light mode implemented. Theme toggle in Settings persists via shared_preferences.

## Phase 1 — P1 Features

- ☐ Section CRUD inside Lists. Tasks assignable to Sections via drag-and-drop or context menu.
- ☐ Subtask list inside Task Detail: add, complete, reorder, delete subtasks.
- ☐ Priority flag (None / Low / Medium / High) on tasks. Colored flag icon on task cards.
- ☐ Reminder creation: absolute time or offset before start time. Stored in reminders table.
- ☐ flutter_local_notifications configured with Android notification channels.
- ☐ Reminder notification fires at remind_at time with task title in notification body.
- ☐ Reminder cancelled correctly when task is deleted or rescheduled.
- ☐ Habit Tracker screen (SCR-05): list all active habits with daily toggle and streak count.
- ☐ Habit Detail screen (SCR-06): full completion history rendered as calendar heatmap or grid.
- ☐ Habit creation sheet (SCR-07): name, color picker, frequency rule.
- ☐ Streak calculator: correctly computes current and longest streak from habit_completions table.
- ☐ Google Calendar OAuth 2.0 sign-in flow implemented via google_sign_in.
- ☐ Google Calendar events fetched for visible date range and displayed as read-only overlay blocks on timeline.
- ☐ Dark mode theme implemented. Toggle persists correctly.
- ☐ List Management screen (SCR-11): rename, recolor, reorder, delete lists.
- ☐ Trash screen (SCR-12): display trashed tasks, restore, permanent delete, empty trash.

## Phase 2 Features

- ☐ Google Drive OAuth scope extended to drive.file.
- ☐ Backup screen (SCR-13): last backup timestamp, manual backup trigger, restore from backup.
- ☐ Google Drive DB upload: create FluxDone_Backup folder, upload fluxdone.db as binary file.

- ☐ Google Drive DB restore: download fluxdone.db, replace local DB, prompt app restart.
- ☐ FluxFocus bridge: Android MethodChannel configured. FocusBlockRequest intent sent on task creation with start/end time.
- ☐ iOS platform target added. All features validated on iOS simulator.
- ☐ Web platform target added. Calendar and responsive layout validated on desktop browser.
- ☐ iOS build signed and validated via Xcode.

## 11. Technical Success Criteria

Phase 1 is technically complete when all of the following are true:

- All P0 and P1 features pass manual end-to-end testing on a physical Android device (API 26+) with zero network connectivity.
- The SQLite database survives app kill, device restart, and OS-level process termination without data loss or corruption.
- No two feature modules import from each other's internal files. Module isolation is enforced.
- All database write operations use parameterized sqflite queries. No raw SQL string concatenation exists in the codebase.
- Recurring task generation produces correct instances for all five recurrence types across month boundaries and leap years.
- Calendar timeline correctly handles: single overlapping task, three overlapping tasks, task spanning midnight, and tasks with no time set (date-only).
- Local notifications fire within 1 second of the scheduled time on a physical device with the app in the background and with the screen off.
- App performance meets all targets defined in Section 8 on a mid-range Android device (e.g., Snapdragon 665 or equivalent).
- The full database can be backed up to Google Drive and restored to a factory-reset device with zero data loss.
- FluxFocus receives the FocusBlockRequest intent from FluxDone and creates a matching block without requiring FluxDone to know FluxFocus's internal implementation.